Track2Data User Guide

Turn idtracker.ai output into clean, calibrated, analysis-ready tables.

© 2026 Martina Bellio. Track2Data is released under the MIT licence.

Contents

- Introduction: what Track2Data does, how to install it, the quick start
- Part I: The workflow, screen by screen
    1. Project
    2. Sessions
    3. Calibration
    4. Zones (optional)
    5. Metadata (optional)
    6. Preprocessing
    7. Metrics
    8. Processing
    9. Preview
    10. Export
- Part II: Reference
    - Understanding the output files
    - Troubleshooting

Introduction

Track2Data turns idtracker.ai output folders into clean, calibrated, analysis-ready tables. This guide follows the desktop app one screen at a time, using a small demo project (two animals in a circular arena). Each screenshot sits next to the explanation of the screen it shows.

> The screenshots are generated from the real app by `scripts/generate_guide_screenshots.py`, so they match the current version. In the app, **Help ▸ Open user guide** brings you here.

Install and start

```
pip install "track2data[ui]"   # or, from a clone: pip install -e ".[ui]"
track2data-gui
```

Only need the command line? `pip install track2data` gives you `track2data run project.t2d.json` (see `track2data --help`).

How the workflow is organised

The left sidebar lists the stages. A badge shows where each one stands:

Badge	Meaning
✓	complete
⚠	usable, but look at the tooltip (e.g. a session has no stable identities)
✗	must be fixed before you can continue
○	nothing entered yet (optional stages stay ○)

Settings save themselves. There are no Apply buttons: a change is stored a moment after you make it, and again when you leave the screen. **Next** (bottom right) stays disabled, with a tooltip saying why, until Project, Sessions and Metrics are filled in and the calibration is valid.

#	Screen	What you do
1	Project	Create or open a project
2	Sessions	Add idtracker.ai output folders
3	Calibration	Choose how pixels become body lengths or cm
4	Zones	Draw regions of interest (optional)
5	Metadata	Attach treatment, date, … from a CSV (optional)
6	Preprocessing	Gap filling, jump removal, smoothing
7	Metrics	Pick what to compute
8	Processing	Run the pipeline
9	Preview	Check trajectories, diagnostics and metric tables
10	Export	Write CSV / Feather / Excel and load them in R or Python

Quick start (five minutes)

1. **File ▸ New project…**, name it. You land on *Sessions*.
2. **Add Folders…** and choose one or more idtracker.ai session folders.
3. Leave *Calibration* on **Body length** unless you know your pixels-per-cm.
4. On *Metrics*, pick a preset such as **Standard locomotor**.
5. **▶ Run pipeline**, then look at **Preview ▸ Trajectories** to confirm tracking looks right.
6. On *Export*, tick the formats you want and **Export**.

Do step 5 before trusting any number: the trajectory view is the quickest way to spot identity swaps, tracking gaps and over-aggressive smoothing.

Part I: The workflow, screen by screen

1. Project

A project is one `.t2d.json` file next to a folder of results. It stores every setting on the following screens, so you can close the app and pick up later, or re-run the identical analysis from the command line.

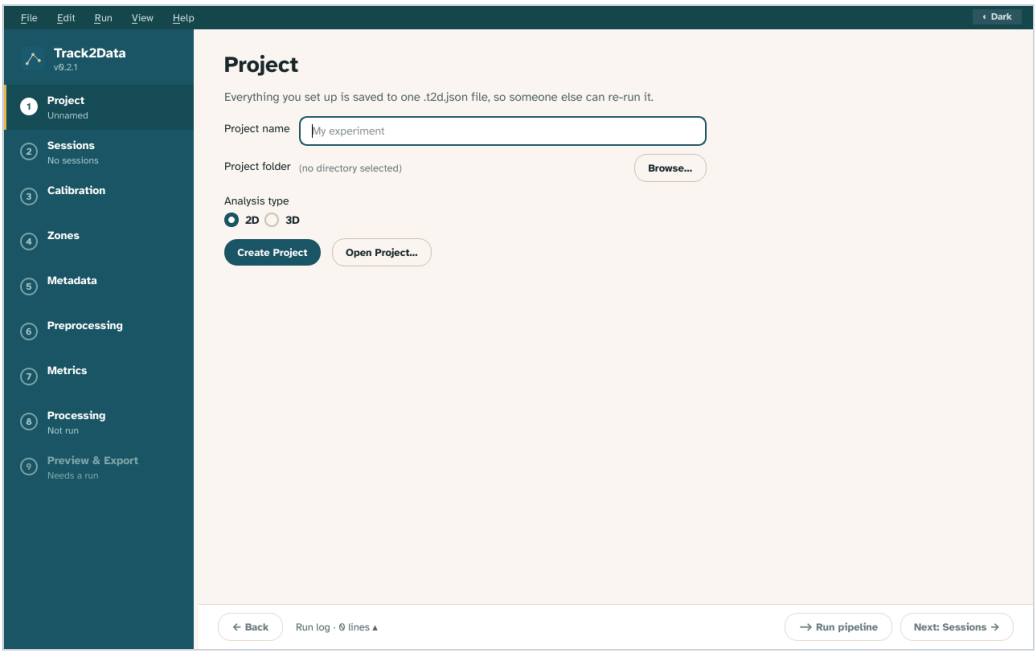


Figure 1. The Project screen, where every analysis starts.

Do this

1. **File ▸ New project…**, enter a name, choose a folder. Or **File ▸ Open project…** for an existing `.t2d.json`.
2. Track2Data moves straight on to Sessions.

Notes

- **Changes are saved automatically** about half a second after you stop editing. The footer says **Unsaved changes** while one is pending and **Saved** once the file is on disk; if writing fails it says **Not saved** (hover for the reason) and your edits stay open. **File ▸ Save project** saves at once, including any value you have typed but not yet confirmed. The file is replaced atomically, so a failed save never leaves a half-written project.
- Closing the window, **New project** and **Open project** save first, and do nothing if saving fails. A reopened project saves back to the file it came from, and **New project** will not overwrite an existing project of the same name.
- Reopening a project re-reads each session folder in the background, so frame counts and calibration readiness appear again after a moment.

- Preprocessed sessions are cached in `.t2d_cache/` inside the project folder. It is safe to delete (`track2data cache clear --cache-dir <dir>` does it too); it is rebuilt when needed.
- **2D or 3D.** The **Analysis type** choice sets whether the project is a 2-D analysis (the default) or a 3-D one. 3D also asks how the recording was made: **One video, two panels** (top and side views in one frame) or **Two videos** (top and side tracked separately). The choice locks after the first session is added; remove all sessions to change it. A 3-D project runs once at least one top/side pair is set up for fusion (see Running a 3-D project); until then Processing, Preview and Export say "Pair and fuse a top and a side session first". Calibration, Zones and Metrics apply to the top-view tracks of each fused pair.

2. Sessions

Add the folders your tracking software wrote its output to, one per recorded video. Adding is two steps: Track2Data looks inside what you picked, then asks you to confirm before anything joins the project.

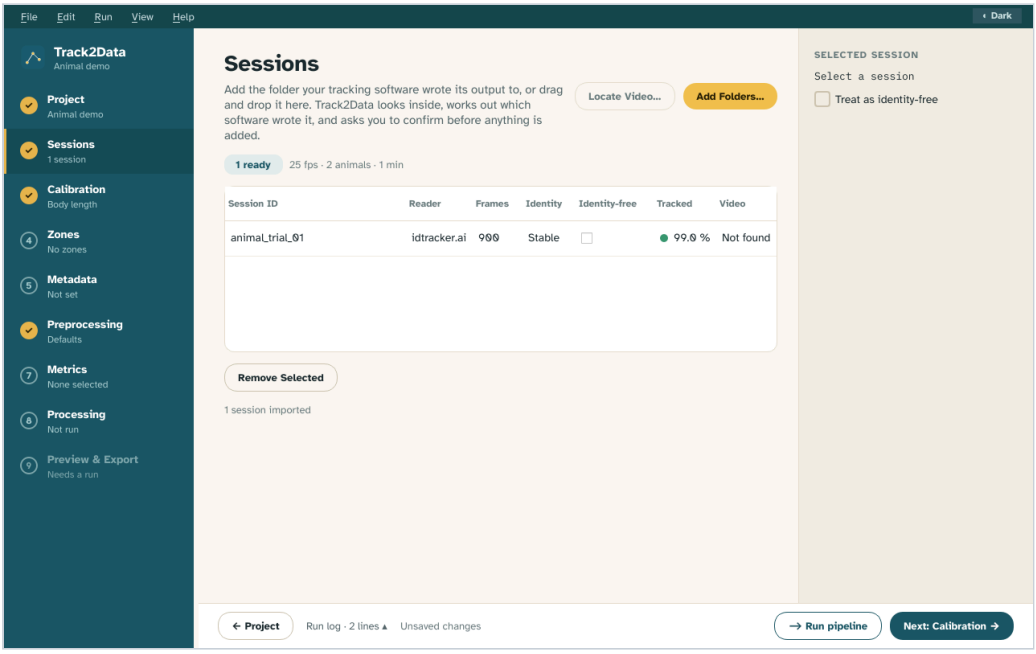


Figure 2. The Sessions screen lists every session with what was read from it.

Add sessions: scan, confirm, add

1. Press **Add Folders…**, or drag folders (or single tracking files) onto the screen. A progress bar with **Cancel** shows while Track2Data looks inside. Picking something else meanwhile replaces the scan.
2. When the scan ends, **Confirm tracking software** opens (Figure 3). It says which software wrote the folder, how sure it is (*High*, *Medium* or *Low confidence*) and why. If it chose wrongly, change **Software**.
3. Fill in any options the files do not record (a frame rate, a frame size, which keypoint stands for the animal). A required option shows *required* until you set it; Track2Data never guesses one, because a made-up frame rate would corrupt every speed.

4. Untick sessions you do not want, or rename them in the *Session* column. **Add** (it shows how many) stays disabled until adding would work, and its tooltip says what is missing.

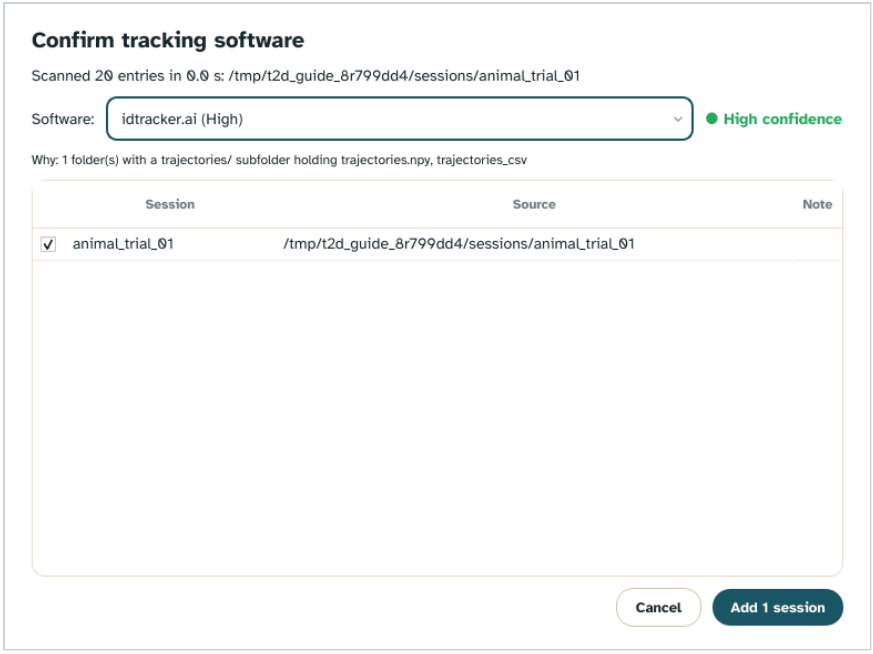


Figure 3. The confirm step: the software found, why, and the sessions about to be added.

A scan that finds nothing still opens the dialog and lists what it saw and what this version can read. A scan that fails writes its reason under the buttons. The chosen software is saved with each session, so reopening the project reads it the same way again.

Each row of the table shows what was read: the software (*Reader*), frame rate, number of frames, number of animals, **Identity** (`Stable` or `Unstable`) and **Video**.

Video: Found, Not found, Located

idtracker.ai records the path the video had on the computer it was tracked on, which is often not valid on yours. **Found** means the file is there. **Not found** means it is not; metrics are unaffected, but the video preview is unavailable. Select the row and press **Locate Video…** to point at the file; the row then reads **Located**, and the choice is stored in the project.

Identity-free

Tick this if the video was tracked *without* identification, or if identities swapped so often that "animal 1" is not one animal. Sessions flagged this way (by you or by idtracker.ai's own `track_wo_identities`) skip every metric that follows an individual across frames, because those numbers would be meaningless. Group metrics and the pooled zone-occupancy metrics still run. The sidebar shows ⚠ while any session is identity-free.

Good to know

- Folders are only read, never modified.
- Adding the same folder twice is ignored.
- The sidebar shows the session count under **Sessions**, and the chips above the table split them into ready and identity-free.

- Supported: idtracker.ai 6.x output (the legacy v5 layout is also read). v4 is not supported yet (Track2Data tells you so if it recognises a v4 folder; see sending a v4 sample).

Views (3-D projects)

In a 3-D project (set on the Project screen) every fish is tracked in two views, so Track2Data has to know which session is the top view, which is the side view, and which top session goes with which side session. This is the **Views** page. It appears only in 3-D projects, under the **Sessions** row of the sidebar, and **Next** and **Back** pass through it between Sessions and Calibration. In 2-D projects it does not exist.

- **Roles.** Each session has a **View** selector: *(not set)*, *Top* or *Side*.
- **Pair by session name.** Two pattern fields, **Top sessions** and **Side sessions**, find the pairs from the session names. A pattern is a regular expression, searched anywhere in the session id, with a group called `key` that marks the part shared by both views. Sessions whose `key` is equal become a pair. Example: `(?P<key>.+)_top$` matches `trial01_top` with key `trial01`, and `(?P<key>.+)_side$` matches `trial01_side` with the same key, so the two are paired. The ⓘ button next to each field explains this in the app. A line under the fields says how many sessions each pattern catches and lists unpaired or ambiguous ones (several sessions with the same key are not paired) and sessions that match **both** patterns ("Match both patterns"; these are not paired). An empty `key` means no match. Press **Pair by pattern** to set the roles and create the pairs. Pressing it again keeps the pairs it made earlier that the patterns still produce, with their matching, and removes the ones they no longer produce. Sessions in pairs you made or changed by hand are skipped: the button never touches those pairs.
- **Pairs.** One row per pair, with a **Same IDs** tick, a **Status** and **Remove**. To pair by hand, choose a top and a side session below the list (only sessions with that role and not yet in a pair are offered) and press **Add pair**. A session belongs to at most one pair.
- **Same IDs.** Tick it when fish with the same label are the same fish in both views; the fish map is then filled in for the labels present in **both** views (the others stay unmatched), and filled in again if the labels of either session change. The tick is disabled while either session has no stable identities or its fish labels are not known yet (hover over it to see which). Unticking clears only the tick and keeps the map. Changing a fish in the matching table clears the tick and makes the pair yours, so **Pair by pattern** no longer touches it; ticking or unticking does too.
- **Match fish.** Select a pair to get a table with one row per top-view fish and a dropdown for the side-view fish (or *(no match)*; a side fish already used is marked). Selecting a row highlights that fish and its match in the two track plots beside the table.
- **Status.** The fish map is checked first. If the checks find a problem, the status shows it and the tooltip lists all of them, for example "cannot match fish: this session has no stable identities" (identity-free sessions cannot be matched), "duplicate side fish: ...", "unknown top fish: ..." or "3 top fish vs 2 side fish" (an unequal count is shown even before any fish is matched). Only when nothing is wrong does the status read *Needs matching* (no fish matched yet) or *Matched* (at least one fish matched; the tooltip lists fish still unmatched).
- **Sidebar.** In a 3-D project the **Sessions** row of the sidebar shows the worse of the Sessions and Views statuses, and its tooltip adds the Views message (for example "Views: Session t1_top is not paired."). Its summary line counts sessions and pairs. The

row keeps its number, without the ✓, until every session has a view; it gets the ✓ once every session has one, and the tooltip still lists what is left (an unpaired session, a pair to match, a session without stable identities). The sidebar checks the saved roles and pairs only; the page checks the fish maps against the fish labels. None of this stops you from continuing.

This only records the correspondence. To run the project, set each pair up for fusion (next sections); see Running a 3-D project.

Panels (one video, two views)

If you chose the layout **One video, two panels** on the Project screen, the top and side views are two parts (panels) of one video. The **Views** page then has a **Panels** section with one row per session, showing its panel (*whole video* or the rectangle as x, y, width × height in pixels). Select a row to use the buttons.

- **Split into panels…** is for one tracker run on the whole frame. The editor offers the presets **Left | Right** and **Top | Bottom** with a split slider (50% by default), exact x, y, width and height fields for each panel, and a choice of which panel is the top view. A preview shows the tracker's background image, or else the first video frame, or else the tracks, with the two rectangles over it. A table lists each fish with its panel, the share of its positions inside (*Inside*) and a *Flag*: *low* under 90%, *left out* under 50% (such a fish is not in that panel's session), *no data* when it has no position. **OK** is off while a rectangle is invalid or a panel would have no fish. The session is replaced by two, `<id>__top` and `<id>__side` (a number is added if the name is taken), each with its panel, the view set, and a hand-made pair.
- **Set panel…** is for two tracker runs on the same video, each limited to one panel: it opens the same editor with one rectangle for the selected session.
- **Clear panel** puts the session back on the whole video.

A session with a panel keeps the fish with at least 50% of their positions inside it, and positions outside become empty (NaN) because they belong to the other view. Coordinates are relative to the panel (its top-left corner is 0, 0) and the video size becomes the panel size, so the Zones, Calibration and Preview screens show only the panel. The same folder with a different panel counts as a different session. Changing or clearing a panel resets the fish matching of its pair (the pair stays).

Zones are still one set per project, and per-view pixel scale calibration is not part of this version.

Fusion (top and side views)

Once a pair has its fish matched, the **Fusion** box on the Views page combines the two views into one session that knows both where each fish is on the floor and how deep it is. It shows *Fusion setup needed.* until you set the pair up, and **Set up fusion…** opens the editor for the selected pair.

- The editor shows the side view with two lines you can drag: the **water surface** and the **tank floor**. The same two rows can be typed. Enter the **tank height** in cm (the water depth between the lines; 20 cm to start with), which gives the side view its scale. If the

side camera is mounted upside down, tick **The side view is upside down**: the lower line then is the water surface, and the labels of the lines and rows follow.
- **Shared horizontal axis** says which top-view axis (x or y) the side view's left-right direction follows; tick **Flip** if it runs the other way. A line under them reminds you that these two only check that the views agree and do not change the fused positions.
- **Frame offset** shifts the side recording against the top one, in frames (side = top + offset). With the layout "One video, two panels" it is hidden and fixed at 0. **Suggest offset** looks for the shift that makes the two views agree best, within ±5 seconds around 0 only; it needs a calibrated top view (set the scale on the Calibration page; the button is greyed out until then), can take about 3 seconds on a 1-hour recording with 5 fish, and answers "no better offset found" when nothing clearly beats 0.
- The summary under the fields updates as you edit: shared frames, fish fused, positions outside the water column, and the agreement. **OK** is off while the settings are invalid or the pair cannot be fused.

Depth is 0 at the surface line and 1 at the floor line; a position outside the lines becomes empty and is counted, never clipped. The cameras are assumed to be at right angles, with no camera calibration. The agreement check compares the side view's horizontal position with the chosen top-view axis in cm (after removing the constant shift between them) and warns when the difference is more than 10% of the top-view range; it needs a calibrated top view, and says so when it cannot run.

Fusion needs both views' fish to be matched, and the two recordings' frame rates must match (within 0.1%). Frames missing from either view are left out, and fish that are not matched are not fused. Until a pair is set up, the Views step in the sidebar shows a warning "Fusion setup needed for ..."; it does not block Next. A pair with fusion settings that fuses without error is run as one fused session; see Running a 3-D project.

Native 3-D files (Anipose, DeepLabCut 3-D)

Some trackers deliver 3-D positions directly, in physical units and with a vertical axis, so there is nothing to pair or fuse. Track2Data reads two of them:

Software	Files	Reader id
Anipose	`pose-3d` CSV (`<keypoint>_x`, `_y`, `_z`, ..., `fnum`)	`anipose`
DeepLabCut 3-D	the triangulated table (three header rows, `x`, `y`, `z` per body part)	`deeplabcut_3d`

Both readers were written from the published layouts and tested on synthetic files only (they are marked *synthetic only* in the confirm step); compare a few positions with the tracker's own output before relying on them.

- **Leave Analysis type on 2D.** A native 3-D project is an ordinary 2-D project whose sessions come from these readers. It has no Views or Fusion step.
- **Options in the confirm step.** `fps` and `unit` are required and never guessed: the files record neither, and a wrong unit scales every distance and speed. Anipose proposes `mm` (the usual board unit) but you must confirm it; DeepLabCut 3-D proposes nothing. The other options are `target_unit` (the project's length unit, default `cm`), `keypoint` (which keypoint stands for the animal; never an average), `likelihood_cutoff` (Anipose only,

applied to the `_score` columns; it needs a score for every keypoint, otherwise set it to 0), `vertical_axis` (`x`, `y` or `z`, default `z`: which axis of the file points up), `flip` (tick when it points down) and `keep_skeleton`. The other two axes become the horizontal plane in cyclic order after the vertical one (`z`: x, y; `x`: y, z; `y`: z, x), which keeps the plane right-handed seen from above, so headings and turn directions have the same sense whichever axis the file calls up. `flip` reverses the vertical axis only.

- **Units are converted when the file is read**, from `unit` to `target_unit`. The first native file you add sets the project's calibration for you: **Session calibration**, the unit label set to the target unit and marked confirmed. Files added later must use the same target unit; a different one is refused (`NATIVE_3D_UNIT_MISMATCH`, naming both units). They must also share the **vertical axis**, because one water column serves every session: a file read with another `vertical_axis` is refused (`NATIVE_3D_AXIS_MISMATCH`, naming both axes); `flip` may differ. The `_cm` column suffix is nominal, as for any project calibrated in another unit; the run's `README.md` names the real unit.
- **No mixing.** A project holds pixel-tracked sessions or native 3-D sessions, not both, and native files cannot join a 3-D (fused) project: the add is refused with `NATIVE_3D_MIXED`, naming the session and both kinds.
- **One animal per file.** Neither format has an animal axis; several animals are several files, hence several sessions.
- **Missing values.** Anipose: an empty cell (NaN) in x, y and z together. DeepLabCut 3-D: a row where x, y and z are all exactly 0; a single 0 is a real coordinate. A gap in Anipose's `fnum` is not filled: the session stores the frames the file has and records the gaps as tracking intervals.
- **Horizontal plane and vertical axis.** The two axes that are not vertical become x and y for the 2-D metrics (IL-1, IL-2 and the rest); the vertical one is the absolute z. z is not smoothed or interpolated.
- **Zones and background are not available** (there is no video frame). The metrics that need an arena or a pixel-sized threshold (IL-3, IL-9, IL-10, IL-14) and all group metrics are not offered (see 3-D metrics).

From the command line: `track2data add PROJECT FOLDER --option unit=mm --option fps=25` (add `--option target_unit=cm --option vertical_axis=y` as needed). Every native file is read in full when it is added, so one unreadable file stops the add.

3. Calibration

Calibration decides which units distances and speeds are reported in. Figure 4 shows the default, **Body length** mode.

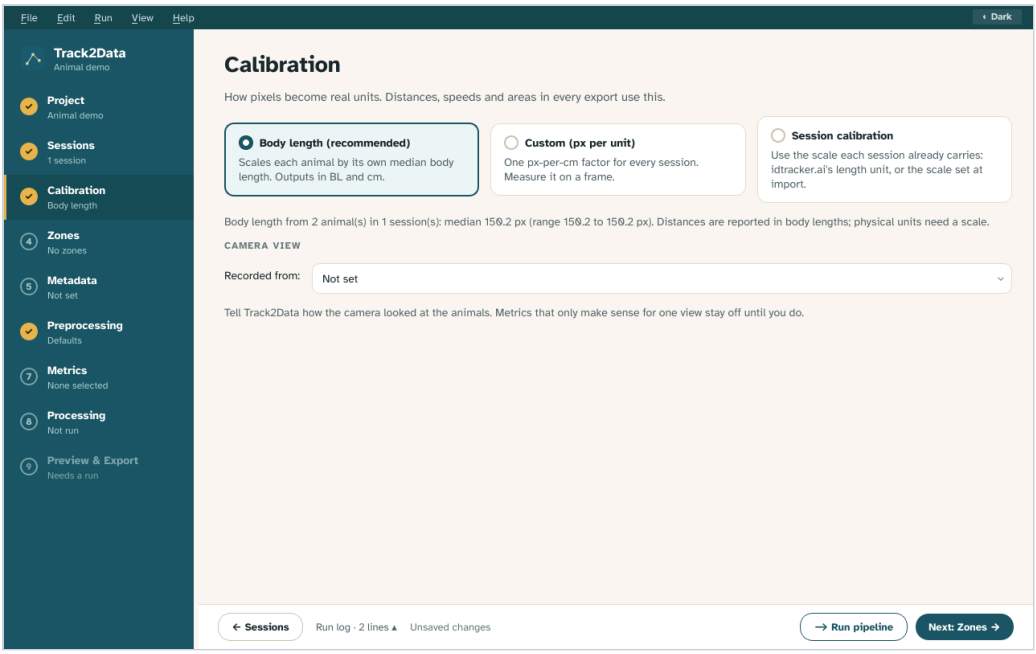


Figure 4. Calibration in Body length mode, with a summary of the body lengths read from your sessions.

Mode	Use it when	Reports
Body length (recommended)	You do not have a reliable scale	`*_bl` columns: distances in body lengths. Physical `*_cm` columns stay empty.
Custom (px per unit)	You know the scale	`*_cm` columns, and `*_bl` columns too whenever the tracker recorded body length
Session calibration	Each session already carries a scale: idtracker.ai's *Length Calibration* tool, a Ctrax / JAABA `trx.mat` `pxpermm`, or a scale you gave when importing	each session's own ratio; you must confirm the unit

In Session calibration mode, Track2Data also records how much your length-calibration clicks disagreed (a relative spread, saved in `sessions.csv` as `length_calibration_rel_sd`) and warns when it is above 5%. A warning means the scale is less certain, and so are the `*_cm` values.

In Body length mode the screen summarises the body lengths read from your sessions (median and range, in pixels) so you can sanity-check them.

Custom scale: measure on the frame

Pick **Custom**, then **Measure on frame…**. Click both ends of something of known length (a ruler, the arena diameter), type its real length, and press OK. The pixels-per-unit value is filled in for you. A third click starts the measurement over.

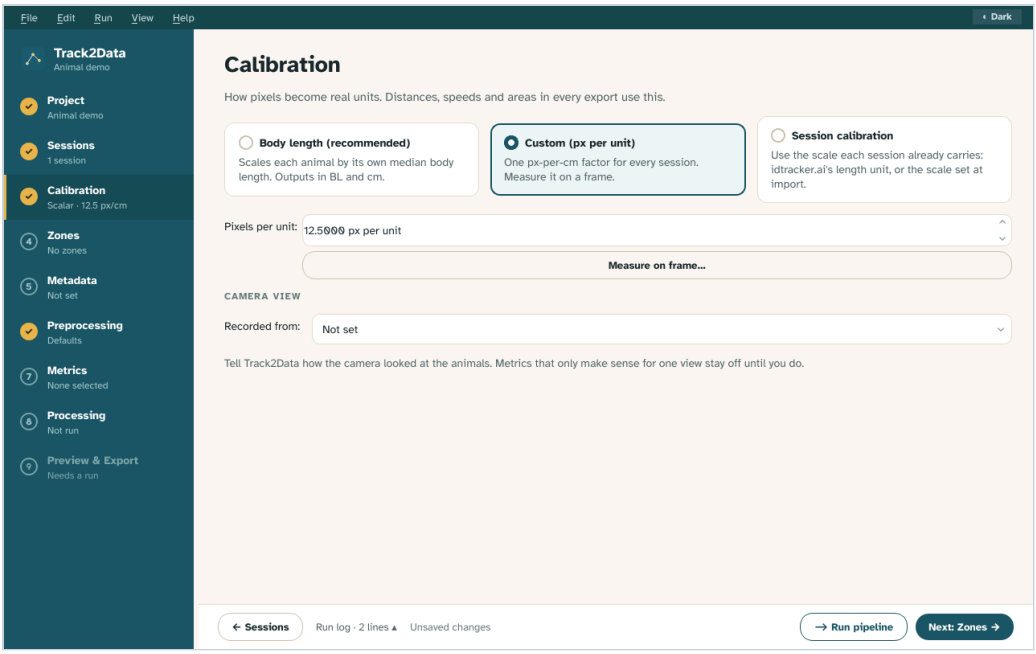


Figure 5. Custom calibration: click both ends of an object of known length to fill in the scale.

If a mode is incomplete (no scale, or an unconfirmed unit) the stage shows ✗ and **Next** is disabled until you fix it.

Session calibration: where each tracker's scale comes from

Only some trackers record a scale. The ⓘ button beside the description opens a short note on what each tracker in your project records, with links to its documentation. Figure 6 shows the mode.

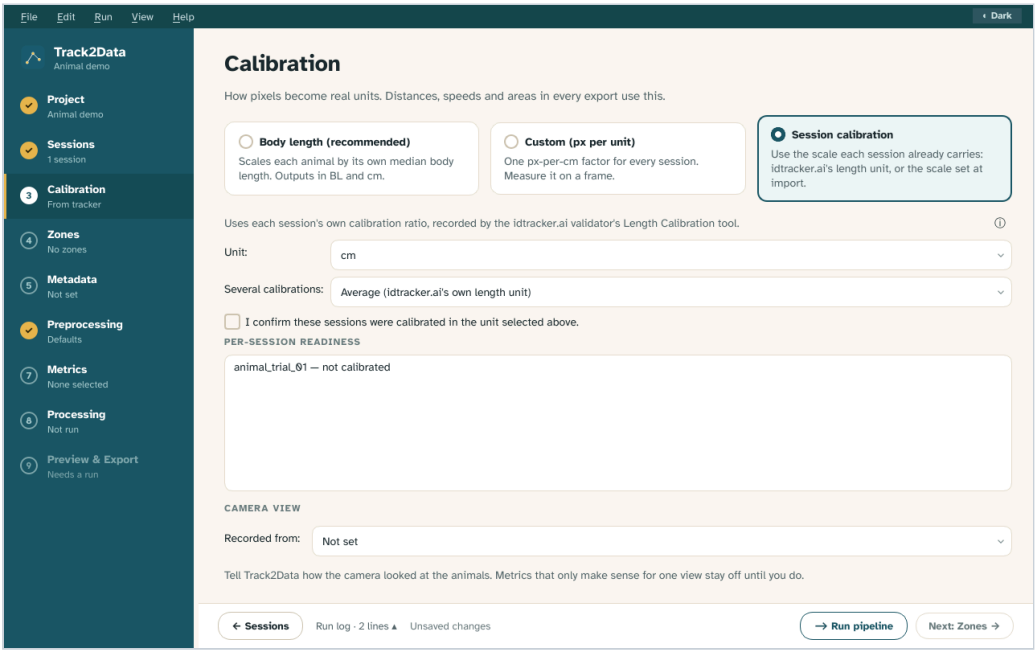


Figure 6. Session calibration: the unit, the choice between several calibrations, the confirmation and each session's readiness.

Tracker	Where the scale comes from
idtracker.ai	The Validator's *Length Calibration* tool saves one or more measurements. `length_unit` is their average: a factor from pixels to *user-defined units*. idtracker.ai does not record whether you measured in cm or mm, so the **Unit** you pick is your own declaration and you must confirm it. See the idtrackerai.Session reference.
Ctrax / JAABA `trx.mat`	`pxpermm` (pixels per millimetre), present once the file's units were converted. Track2Data turns it into pixels per cm and ignores a value of exactly 1, which those tools use for "never calibrated". See the Ctrax trx fields.
DeepLabCut, SLEAP, Ctrax raw `.mat`	Their files record no scale. Give **Scale (pixels per cm)** in the confirm step when you add the sessions, or use **Custom** or **Body length**.

The per-session readiness list says whether each session is calibrated and with which value. A session without a scale reads *not calibrated* (idtracker.ai) or *no scale set at import* (other trackers). The **Session calibration** card is disabled when no session in the project can supply a scale.

Several calibrations. An idtracker.ai session can hold several length measurements. With **Several calibrations** you choose **Average** (idtracker.ai's own `length_unit`, the default) or **Median** (one badly placed click cannot move it). The list shows which one is in use, for example *median of 3*. If the saved `length_unit` differs from the average of the session's measurements (they were edited after export), the saved value is used and the run log carries a warning.

Camera view

Further down the same screen, **Camera view** says how the camera looked at the animals: **Not set** (the default), **Top-down**, or **Side view**. It is saved with the project, separately from the calibration, and a project that never sets it behaves exactly as before.

Some metrics only make sense for one view. Today that is **Vertical Position (Depth)**, IL-15, which needs a side view: on a top-down recording the image's vertical axis is not depth. Until you declare a side view, that row is greyed on the Metrics screen and its tooltip says how to switch it on; if it is selected anyway, the run skips it and the run's README says so. Declaring a view does not switch off any existing metric. Thigmotaxis (IL-14) and distance from the centre (IL-3) still assume you are looking down on the arena, so read them accordingly on a side view.

For depth, draw a **main** zone from the waterline to the floor on the Zones screen. The top edge is the surface and the bottom edge the floor; Track2Data never substitutes the video frame, so with no main zone the depth columns are empty and say why. For time in the upper or lower part of the tank, draw stacked secondary zones and use the zone metrics (Z-1 to Z-9).

Water column (native 3-D files)

A native 3-D project shows a **WATER COLUMN** group with two fields, **Surface z** and **Floor z**: where the water surface and the floor lie on the file's own vertical axis (as the file writes it, before any **flip**; Track2Data applies each session's flip itself), in the project's length unit (either order is accepted; they must be finite and different). You enter them; Track2Data does not

estimate them from the data. With both set, each session gets a depth (0 at the surface, 1 at the floor, `(z - surface) / (floor - surface)`) and a water height (`|floor - surface|`), which is what **IL-15** needs. A position outside the column (above the surface or below the floor) has no depth: it is left empty and counted in IL-15's `frac_outside_extent`, never clipped to the edge, and it is also left out of IL-16 and IL-17 (the step touching it is dropped). The same surface and floor apply to every session, so the files must share a vertical axis and an origin (the origin cannot be checked from the files). Without the water column IL-15 is greyed out ("needs the water column (surface and floor z) on the Calibration page"); IL-16 and IL-17 do not need it, because z is already absolute; they use the column only to leave out samples outside it.

A native 3-D project must stay in **Session calibration** with the unit label equal to the target unit and confirmed; the project is blocked otherwise (`NATIVE_3D_NEEDS_SESSION_CALIBRATION`, or a message asking you to confirm the unit or to match the sessions' unit). The mode is set for you when you add the first file, and nothing else (import, validate, run) changes it.

4. Zones (optional)

Zones are regions of interest (the whole arena, a centre area, a feeder). Zone metrics need at least one. You can:

- **Import ROIs from Session**: use the polygons drawn in idtracker.ai's validator.
- **Load zones from CSV…**.
- Draw your own, as described below.

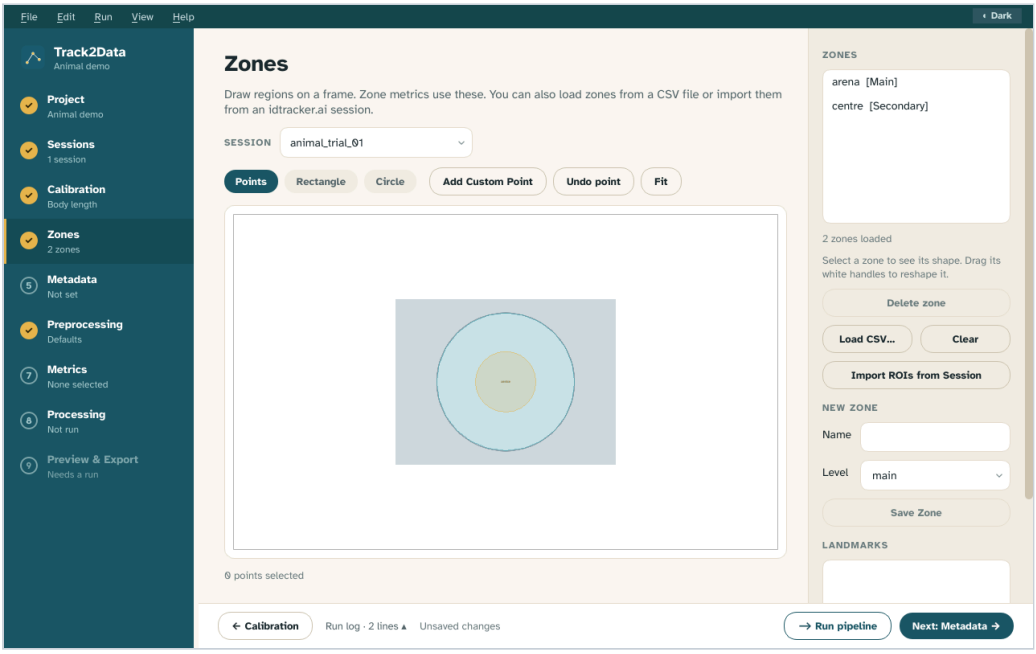


Figure 7. The zone list, with the import and load buttons.

Drawing zones

Scroll down to the canvas, which shows the session's background image. Saved zones appear shaded with their names.

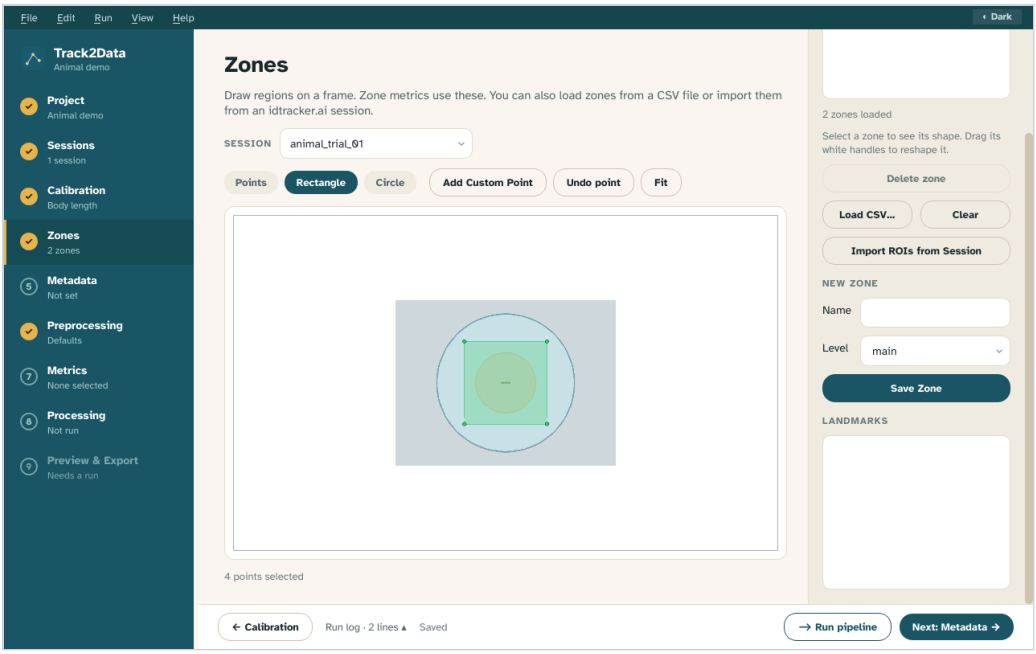


Figure 8. The zone canvas: saved zones are shaded and named, validator landmarks are blue.

Tool	How
Points	Click validator landmarks (blue) in order. **Add Custom Point** lets you click anywhere.
Rectangle	Drag from one corner to the opposite corner
Circle	Drag from the centre outwards
Undo point / Ctrl+Z	Remove the last vertex
Fit	Fit the whole image in view. The mouse wheel zooms; hold the middle button and drag to pan.

Custom vertices (orange/green) can be dragged to fine-tune the outline. When the shape looks right (at least 3 points), give it a **name** and a **level** (`main` or `secondary`) and press **Save Zone**.

To reshape a saved zone, select it in the list and drag its vertices. A move that would leave the zone with no area (all vertices in a line), repeat a point, or make its edges cross is refused: a message says why, the handle jumps back and the zone stays as it was.

If a zone's source resolution differs from a session's video, a yellow warning appears.

5. Metadata (optional)

Attach experimental information (treatment, date, tank, …) to every row of the output.

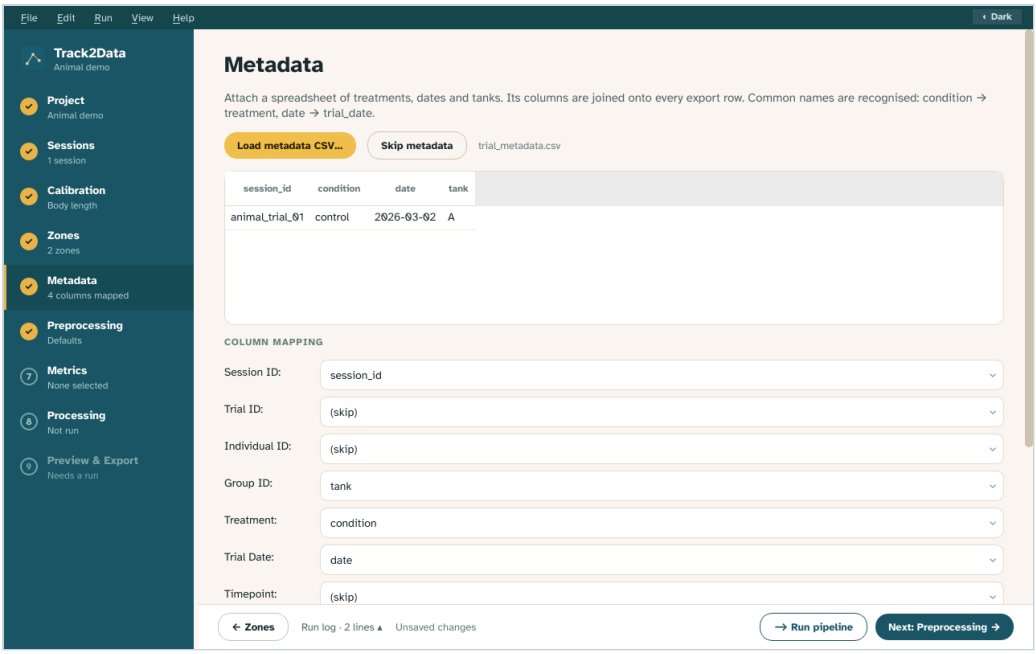


Figure 9. The Metadata screen: CSV preview, column mapping and the match summary.

1. **Load metadata CSV…**. The first rows are previewed.
2. Match columns to fields. Names like `date`, `condition` or `tank` are matched automatically to *Trial date*, *Treatment*, *Group ID*; change any you disagree with, or leave a field on `(skip)`.
3. The line under the mapping tells you how many sessions found a row (`1 of 1 sessions matched`) and names any that did not.

The `session_id` column must equal the session folder name. Without an *Individual ID* column one row is matched per session (if several rows match, the first is used and the summary says so).

Per-animal information (sex, weight, genotype of individual animals)

Give the CSV **one row per animal** and map its animal column (`fish_id`, `animal_id` or any name) to *Individual ID*. Then:

- **Match animals by** decides how each value is matched to an animal. *Validator label* uses the names set in the idtracker.ai Validator (the default labels are `1`, `2`, …) and falls back to the 0-based position for a session that has none. *Position* always uses the 0-based position (the `individual_id` of the exported tables).
- Tick the other columns you want under **Also include these columns**. Columns that are not mapped to a field and not ticked are dropped.
- Per-animal values land on every row that has an `individual_id` (the per-frame table and the individual metric tables). Values that are the same for all animals of a session (such as `treatment`) also go onto the group tables; per-animal values never do.
- The summary lists animals with no row (their cells stay empty), and CSV rows whose animal matches nothing.
- **Identity-free sessions** get no per-animal values: their rows are detection slots, not animals. Session-level fields still apply.

Pick the right match mode: if your CSV counts animals from 1 and the Validator labels are the default `1`, `2`, … use *Validator label*; if it counts from 0, use *Position*. A column named like an engine column (`speed_px_s`, `frame`, `bin_index`, …) is never carried, so it cannot overwrite data.

Skip metadata removes the file if you change your mind.

6. Preprocessing

Cleaning applied to the trajectories before any metric is computed. Defaults suit most data. Every step has an **Enabled** box; the raw data is never overwritten.

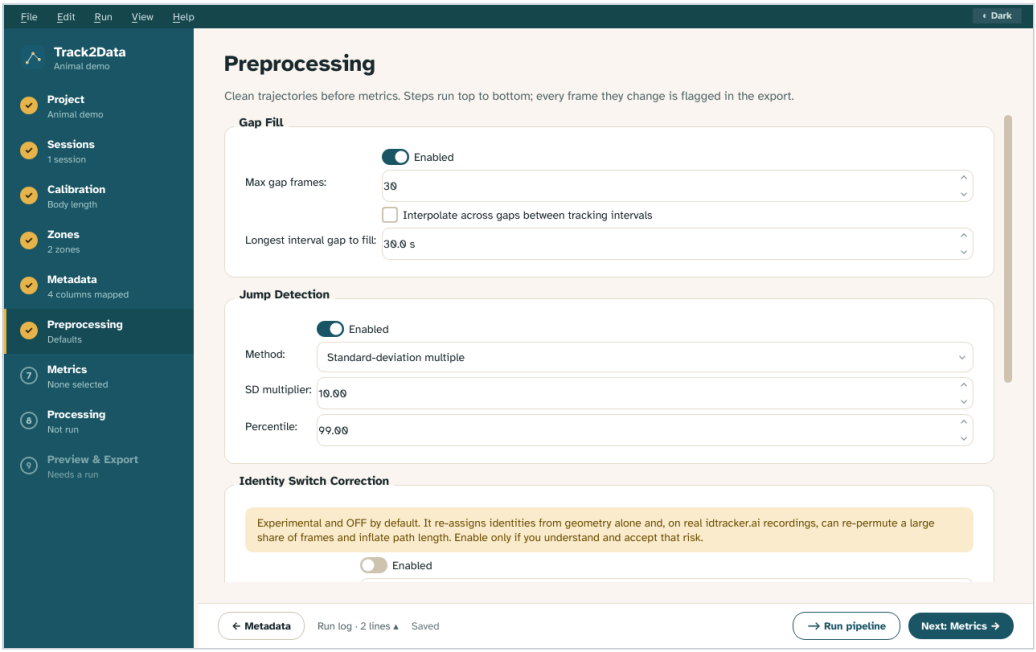


Figure 10. The Preprocessing screen with the default settings.

Step	What it does
Gap fill	Linearly interpolates tracking gaps up to *Max gap frames* (gaps between tracking intervals have their own option, below)
Jump detection	Finds implausible position jumps (standard-deviation multiple, percentile, or idtracker.ai's own velocity threshold) and replaces them
Identity switch correction	See below. Off by default
Smoothing	Moving average or Savitzky–Golay, over *Window* frames
Coverage gate	Warns when more than *Max NaN fraction* of an animal's frames are missing

Sessions tracked in separate intervals

idtracker.ai can track only some stretches of a video. Its trajectory file then holds just those frames, and Track2Data puts them back on the real timeline: speeds, smoothing and zone times are computed over the true elapsed time, and the stretches in between are **never read as adjacent frames** (that used to turn a move across a 40-second gap into one frame of motion).

By default the untracked stretch stays empty: nothing is interpolated, distance is not counted across it and a stay in a zone is cut there. Tick **Interpolate across gaps between tracking intervals** to fill gaps of up to *Longest interval gap to fill* seconds (30 s by default) with a straight line between each animal's last and next observed position. This is an assumption about where the animal went, not a measurement:

- Filled frames appear in the per-frame table with `was_interpolated` true and `in_tracking_interval` false, and are counted by the *Interpolated* column of the quality grid (diagnostic D-11).
- Distance, speed, activity and zone time include them; zone events on them carry `estimated`.
- It needs stable identities and an observed position for that animal on both sides, and never extrapolates. Sessions marked identity-free are never bridged.
- A gap longer than the limit stays a break. The limit exists so one long gap cannot quietly be replaced by a made-up path.

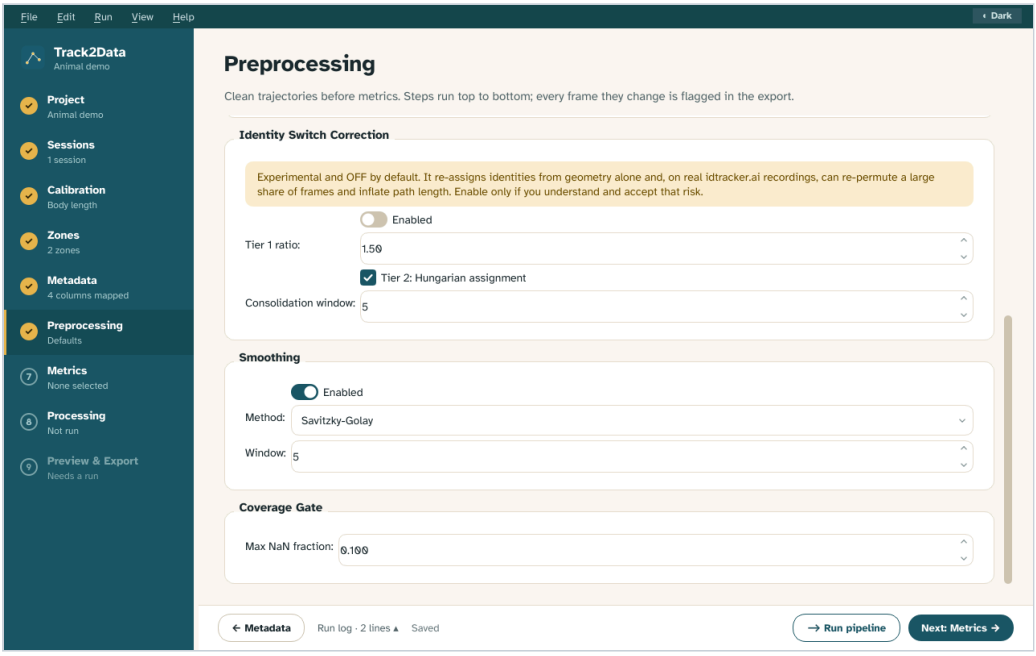


Figure 11. The identity switch correction options (experimental, off by default).

Identity switch correction is experimental and off by default. It reassigns identities from geometry alone, ignoring idtracker.ai's own fragment boundaries. On a real recording it re-permuted a large share of frames and inflated path length enormously. Leave it off unless you have checked the result in Preview ▸ Trajectories with *Raw + processed* shown.

Settings you change here are saved automatically and used for the next run.

7. Metrics

Choose what to compute. Metrics are grouped on three tabs: **Individual** (per animal), **Group** (across animals) and **Zone** (needs zones; the tab is disabled without them).

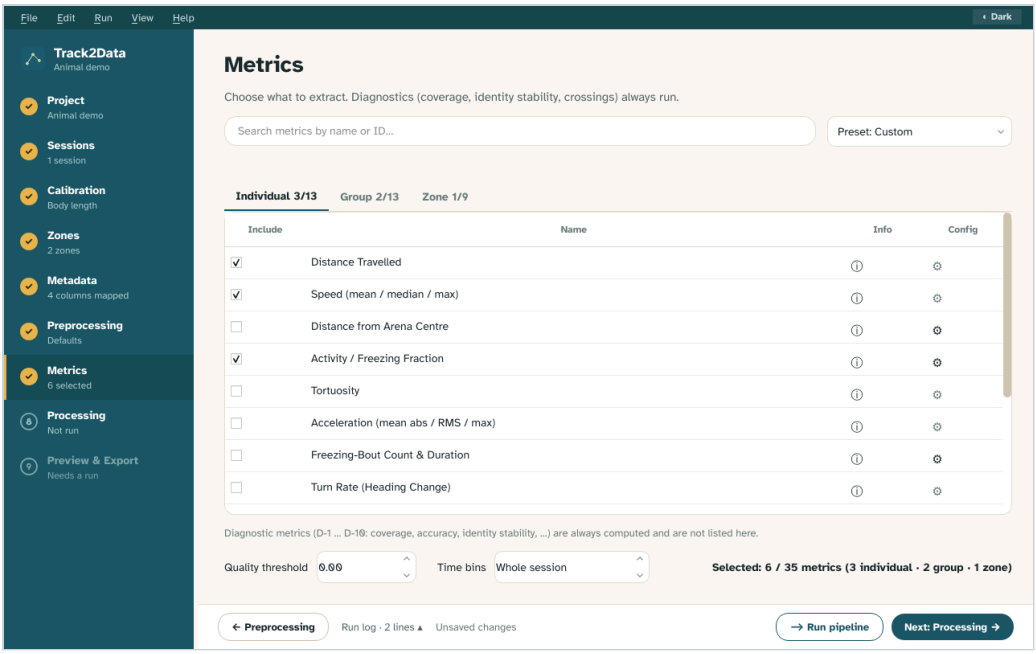


Figure 12. The Metrics screen, with search, presets and the three metric tabs.

- **Search** filters all tabs by name or ID (`speed`, `IL-2`). The line beneath says which tabs match.
- **Presets** replace the current selection: *Standard locomotor*, *Thigmotaxis & space use*, *Social dynamics*, *All metrics*.
- ⓘ opens the definition, formula and literature reference of a metric; ⚙ edits its parameters (available on metrics that have any).
- The counter shows how many are selected.
- **Quality threshold** drops frames whose identification probability is below the value.
- **Time bins** splits every session into bins of the chosen length (minutes) and reports each metric per bin, with `bin_index`, `bin_start_s` and `bin_end_s` columns. *Whole session* (the default) turns it off. Whole-track metrics (tortuosity, home-base stability) stay one row per animal with empty bin columns; diagnostics are always whole-session. Thresholds such as the freezing speed threshold are fixed from the whole session, so bins are comparable.
- Rows are greyed out for sessions without stable identities; see Sessions. The 3-D metrics (IL-16, IL-17, GL-16) are greyed out in a 2-D project and under body-length calibration; see 3-D metrics. A preset never ticks a greyed-out row.

Diagnostics (coverage, tracking accuracy, identity stability, ...) are always computed and are not listed here; they appear in Preview ▸ Diagnostics.

Every metric is documented in `docs/METRICS_SPEC.md`.

8. Processing

Validate pipeline checks the project and lists anything that would stop a run. **Run pipeline** (or the button in the footer bar, or Ctrl+R) processes every session: import, preprocessing, calibration, zone assignment, metrics, export.

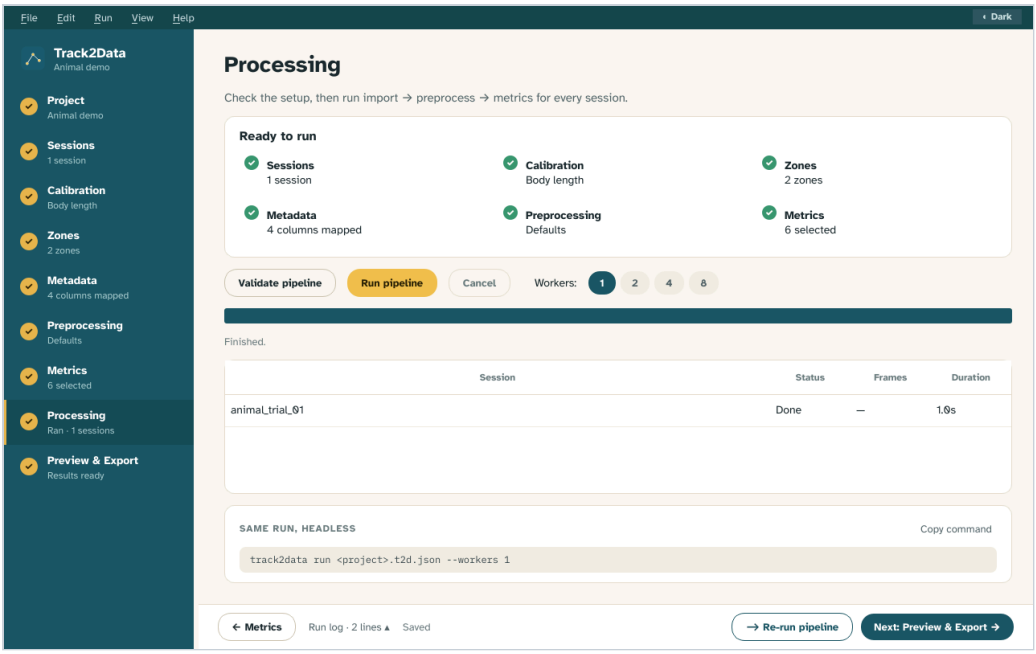


Figure 13. The Processing screen: one row per session, with status and duration.

- The table shows each session's status and duration. A failed session is marked *Failed* and does not stop the others.
- **Workers** sets how many sessions run at the same time. `1` runs them one after another. More workers help when each session takes many seconds or more; for short sessions the start-up cost of each worker makes it slower.
- **Cancel** stops the run at the next safe point (between sessions, preprocessing steps or metrics).
- Preprocessed sessions are cached, so re-running after changing only metrics or export settings skips the slow parts.

Results are written to `<project>/exports/<timestamp>/<session>/`.

Running a 3-D project

In a 3-D project the run works on *fused pairs*, not on single sessions. Each top/side pair that has fusion settings and fuses without an error becomes one **run unit**, named `<top>+<side>` (for example `t1+s1`), and is processed like an ordinary session: its top-view tracks give the usual metrics, and the fusion's depth gives vertical position (IL-15) and the 3-D metrics (see 3-D metrics). Results go to `<run>/<top>+<side>/`.

- **Processing** shows one more check, a status dot followed by `Will run: t1+s1, t2+s2`, then one `Skipped: <session> — <reason>` line per session that will be left out. A skipped session is a warning, not a problem: the other units still run. The line says *Checking which pairs can run...* for a moment, because every pair is fused to check it. If no pair can run, **Run pipeline** is off and the line says why ("Nothing will run: ..."). The page's **Validate pipeline** button reports the same outcome ("✓ 2 unit(s) will run" or "✗ Nothing will run: ...").
- **Skip reasons** are "not in a fusable pair" (the session has no pair), "pair t+s: no fusion settings for this pair", or the fusion error with the prefix "pair t+s: " (for example, the frame rates differ). Both sessions of a skipped pair are listed.

- **Blocked** means: Processing, Preview and Export stay off, with "Pair and fuse a top and a side session first", while no pair has fusion settings. A pair that has settings but does not fuse is reported by the Processing check and at run time, and does not stop the other pairs.
- **Preview ▸ Trajectories** lists the run units; loading one fuses the pair and shows the top-view tracks under the unit's name.
- **Validate** (and `track2data validate`) looks only at the sessions that run: the consistency checks (frame rate, resolution, calibration spread) read the top session of each pair with fusion settings, so a side camera with another resolution is not reported, and session calibration needs a scale on both sessions of such pairs (the side view is preprocessed before fusing), not on the sessions that are skipped.
- **Export** runs the same units. `track2data run` does the same from the command line and prints the skipped sessions; `track2data validate` lists unfusable pairs and unpaired sessions as notes, and fails only when no pair can run.

What the files contain, for a fused unit:

- The per-frame table (`master_fish_by_frame.csv`, the Feather file and the Excel sheet) has two more columns, `depth_fraction` (0 = water surface, 1 = tank floor) and `depth_cm` (the fraction times the tank height; empty when the height is unknown). A frame where the position or the depth is missing has both empty. Two flag columns, `depth_interpolated` and `depth_jump_replaced`, are True where the depth rests on a side-view position that preprocessing gap-filled or replaced after a jump (the depth is kept, not blanked; report how much of a result rests on such frames). The wide CSV also writes `master_fish_by_frame.csv` for a 3-D run, so its depth is not lost. Where the fused frames jump (a frame one of the views lacks), the track keeps a break, as for a 2-D session tracked in separate intervals: no row is written for the missing frames, and path length and the other frame-to-frame metrics do not bridge the gap.
- **Fish numbering.** `individual_id` numbers the fused fish 0, 1, 2, … in the top view's order. When the fish map leaves a top fish out, the ones after it move up (top fish 0 and 2 become fused fish 0 and 1); `individual_label` keeps the top view's identity (`0` and `2`).
- **Metadata.** A unit carries the TOP session's metadata: the row you mapped for `t1` gives the `group_id`, treatment and other columns of `t1+s1` (the side session's row is not used). Per-animal metadata matches the fused fish by the top session's label, or, with matching by index, by the top session's animal index (so in the example above the row for animal 2 goes to fused fish 1).
- **IL-15** takes the depth from the fusion: mean, median and SD of the depth fraction, `mean_depth_cm`, and `frac_outside_extent`, the share of the fish's side-view positions that fell outside the water column (they are left empty in the depth, not clipped). `depth_extent_source` is `fusion`. IL-15 runs whatever the project's camera view is; the other metrics use the top view.
- The unit's `README.md` has a **3-D fusion** section (settings, frames in common, fish fused and left out, positions outside the water column, agreement), and `manifest.json` repeats it under `run_metadata.fusion`.
- `PROJECT_SUMMARY.md` says "Units processed (fused pairs): 2 of 2; 1 session skipped".
- `sessions.csv` has one row per unit, with extra columns for the fusion: `unit_kind`, `top_session_id`, `side_session_id`, the offset, axis, flip, surface and floor rows and tank height, the overlap, fish fused and left out, positions outside the column, the agreement

and its warning or reason it was skipped, and `side_trajectory_sha256` (the checksum of the side view's trajectory file; the usual `trajectory_sha256` is the top view's). A unit that fails after fusing keeps these columns next to its error.
- The unit's README lists the top session's preprocessing steps; the side session went through the same preprocessing settings. The pooled `all_sessions/manifest.json` repeats each unit's fusion under `run_metadata.fusion`.
- If anything was skipped, `skipped.csv` (`session_id`, `reason`) and a **Skipped sessions** table in `PROJECT_SUMMARY.md` list it. Without skips, neither exists. A 2-D project's output is unchanged.

3-D metrics

A fused unit has x and y from the top view and a depth from the side view, so three more metrics can measure in all three directions. The 2-D metrics (IL-1, IL-2, GL-1 and the rest) are unchanged and keep measuring in the top-view plane.

Metric	Columns	Units
IL-16 3-D Distance Travelled	`path_length_3d_cm`, `path_length_3d_bl`, `n_valid_steps`	cm, body lengths, count
IL-17 3-D Speed (mean / median / max)	`mean_speed_3d_cm_s`, `median_speed_3d_cm_s`, `max_speed_3d_cm_s`, `mean_speed_3d_bl_s`	cm/s, body lengths/s
GL-16 3-D Nearest-Neighbour Distance	`mean_nnd_3d_cm`, `median_nnd_3d_cm`, `mean_nnd_3d_bl`, `n_skipped_frames_3d`, `frac_frames_used_3d`	cm, body lengths, frames, fraction

- **Positions in cm.** x and y are the top view's pixels divided by its scale; the depth is the depth fraction times the **tank height** of the pair's fusion settings. Distances are straight lines in (x, y, depth). Body lengths use the animal's body length converted to cm with the same scale (empty when the body length is unknown).
- **Needs a cm scale for the top view.** Use **scalar** calibration (a pixels-per-cm value) or **session** calibration on the Calibration screen. Under **body-length** calibration there is no top-view scale, so the three rows are greyed out ("needs a cm scale for the top view (use scalar or session calibration)"). In a 2-D project they are greyed out too ("needs a fused 3-D session"). Scalar calibration with no pixels-per-cm value set greys them out as well. If a run starts anyway (for example `track2data run`, which warns "Scalar calibration selected but px_per_cm is not set" and carries on), the metrics are skipped for every unit with "needs a cm scale for the top view", listed under **Metrics skipped** in its README; the other metrics still run. Session calibration where the sessions carry no length unit is different: it blocks the whole 3-D run (CAL-SESSION-MISSING), as for any metric. Track2Data does not estimate the scale from the fusion, because that would add an error the outputs would not show.
- **Depth comes from the fusion settings:** the surface and floor rows and the tank height you set for the pair. A wrong tank height scales every vertical distance and speed.
- **Gaps give empty values, never 0.** A step (IL-16) needs a valid x, y and depth at both of its frames; `n_valid_steps` says how many were used. IL-17 uses the frames where the horizontal and the vertical speed both exist. GL-16 skips a frame where any fish lacks x, y

or depth and counts it in `n_skipped_frames_3d`; `frac_frames_used_3d` is the share of real frames it used. GL-1 and GL-16 can use different frames (GL-16 also needs every depth), so `mean_nnd_3d_cm` is not like-for-like with `mean_nnd_cm` when that share is low. An animal (or a group) with nothing valid is empty. IL-16 and IL-17 need identities, as IL-1 and IL-2 do; GL-16 does not.
- **IL-17** takes the horizontal speed from the same speed as IL-2 and the vertical speed from the depth with the same speed estimator as the rest of the run. Over the frames where the vertical speed exists it is at least IL-2's cm/s; IL-16 is at least IL-1 over the steps where the depth is also valid; GL-16 is at least GL-1 for a frame where both exist. Over whole sessions these do not have to hold, because the 3-D metrics leave out frames the 2-D ones keep.
- **Time bins** work as for the 2-D metrics (each bin has its own value). In a binned run the vertical speed is estimated inside each bin.
- **Limits.** Depth comes from a side camera at right angles to the top camera; there is no correction for refraction or parallax. Vertical speed as its own metric, 3-D inter-individual distance and cohesion are not included.

In a native 3-D project the same three metrics run on the sessions' absolute z (no tank height, no fusion): the positions are `(x, y, z)` in the project's unit, and IL-16 and IL-17 need no water column. Example: a fish that moves (3, 4, 12) mm per frame in an Anipose file at 25 frames per second, read as cm, steps hypot(0.3, 0.4, 1.2) = 1.3 cm per frame, so 10 steps give an IL-16 of 13 cm and an IL-17 of 1.3 x 25 = 32.5 cm/s; its 2-D metrics measure the horizontal steps of 0.5 cm (IL-1 5 cm, IL-2 12.5 cm/s). **GL-16 is not offered for native files** (it needs two animals in one session and a file holds one), and neither is any other group metric ("needs several animals in one session; a native 3-D file holds one animal"). **IL-3, IL-9, IL-10 and IL-14 are not offered either** ("uses a pixel-sized default and the project's native 3-D files have no pixel frame"): their default thresholds are in pixels and would silently change with the project unit. The Metrics screen greys them out, `validate` and the run `README.md` list the reason. IL-15 appears once the water column is set. The per-frame table has a `z_cm` column, the file's own vertical coordinate after the same gap filling and smoothing as x and y (so IL-16 can be reproduced from `x_cm`, `y_cm` and `z_cm`); with a water column it also carries `depth_fraction` and `depth_cm` as for a fused unit. The run's `README.md` has a **Native 3-D source** section (reader, file unit, project unit, vertical axis, flip, water column or its absence) and, with a water column, a row **Samples outside the water column** with the count left out.

Limits of native 3-D files: one animal per file; no mixing with pixel sessions; the unit you give must be the one of the calibration board (a wrong one scales everything); no refraction correction; the water column is entered, not estimated; no zones or background image; no group metrics and no IL-3, IL-9, IL-10 or IL-14; a multi-animal DeepLabCut 3-D table (with an `individuals` header row) is named by the scan as one it cannot read yet.

Limits of this version:

- Zones are one set, applied to the top view. There are no per-view or depth-band zones.
- `sensitivity` refuses 3-D projects ("sensitivity is not supported for 3-D projects yet").
- Every run fuses the pairs again; nothing fused is cached.

9. Preview

Look before you export. Four tabs: **Summary**, **Diagnostics**, **Metrics**, **Trajectories**.

Are these results current? If you change a setting that affects the numbers (sessions, calibration, zones, metadata, preprocessing, metrics) after a run, a warning appears at the top of Preview and the sidebar says *Settings changed · re-run*: what you are looking at came from the older settings. Changing only where or in which format to export does not do this, and putting a setting back clears the warning. A run that finishes after you edited is judged against the settings it started with, so it cannot make newer edits look current, and a run that finishes for a project you have since replaced is discarded. A run in which no session succeeded is shown as *No successful sessions*, not as ready. Export recomputes the metrics under the current settings either way.

Trajectories

Choose a session and **Load trajectories** (it reuses the cache after a run). Then:

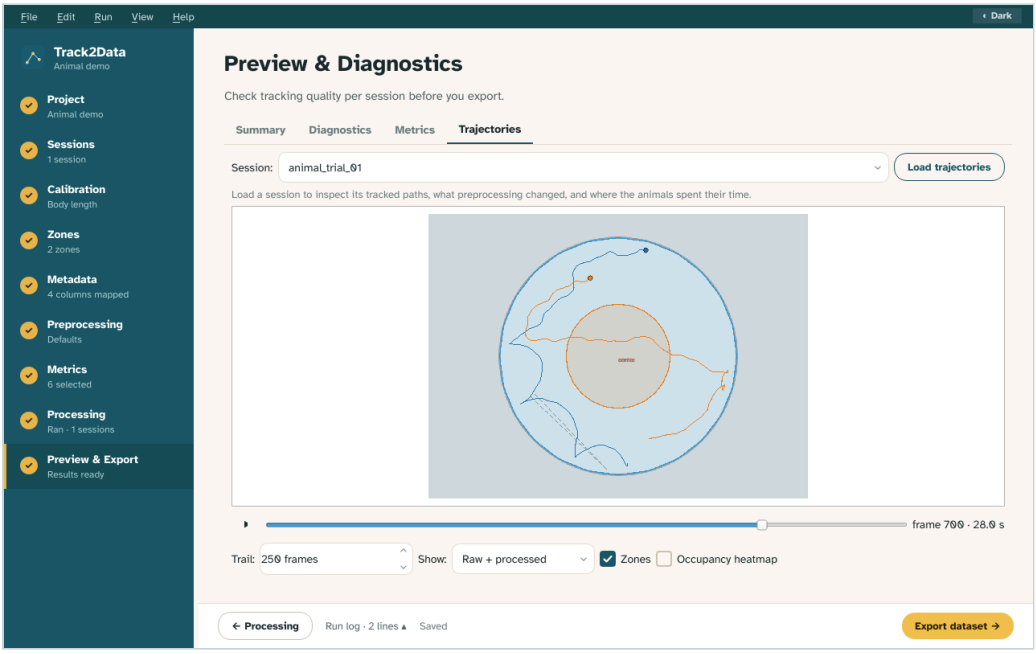


Figure 14. Trajectories with Raw + processed shown: the dashed grey line is a tracking jump that was removed.

- Drag the slider, or press ▶, to move through time. **Trail** sets how many past frames are drawn.
- **Show ▸ Raw + processed** draws the raw path as a dashed grey line next to the processed one, so you can see exactly what gap filling, jump removal and smoothing changed. In Figure 14 the dashed line is a tracking jump that was removed.
- **Zones** shows or hides the saved zones. The mouse wheel zooms.

What to look for: a path that suddenly jumps across the arena (identity swap or detection error); long straight lines where an animal was lost; smoothing that cuts corners of real turns.

Occupancy heatmap

Tick **Occupancy heatmap** to see where the animals spent their time (red = most).

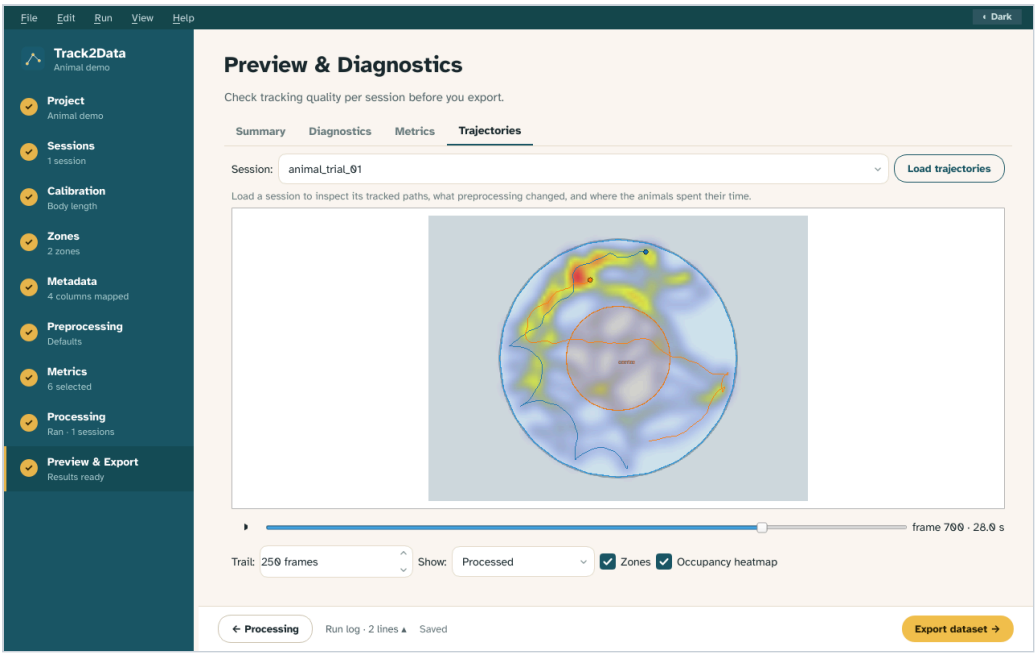


Figure 15. The occupancy heatmap.

Diagnostics

Per-animal tracking coverage and identification probability, session-level accuracy, inconsistent frames and identity stability, plus the list of preprocessing steps and how many frames each one changed.

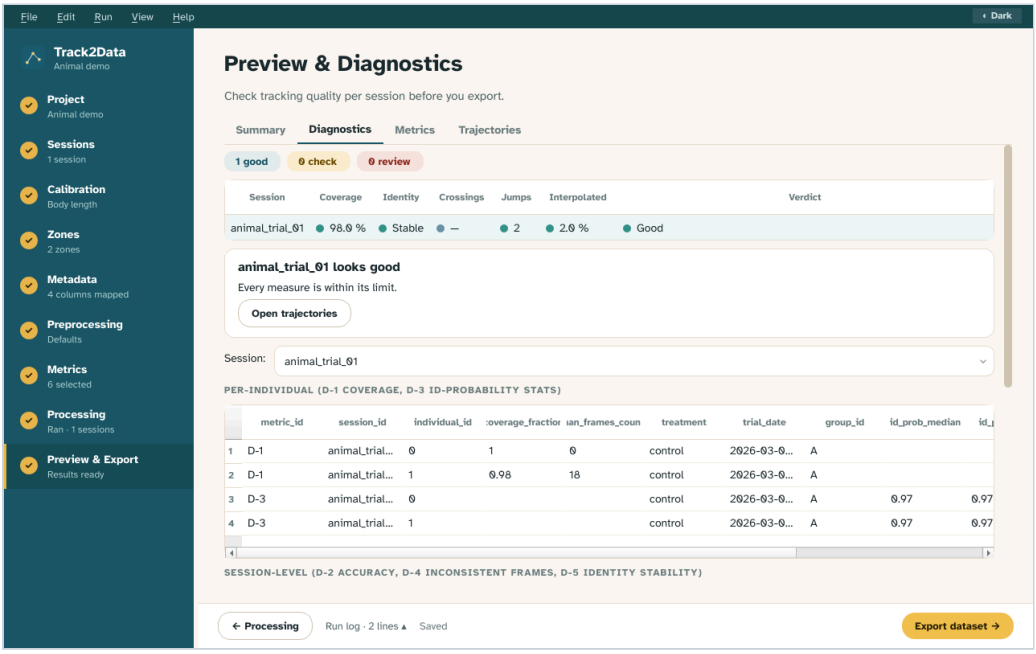


Figure 16. The Diagnostics tab.

Metrics

A table preview of each selected metric for the chosen session.

10. Export

Tick the formats you want, choose where to write them, and press **Export**.

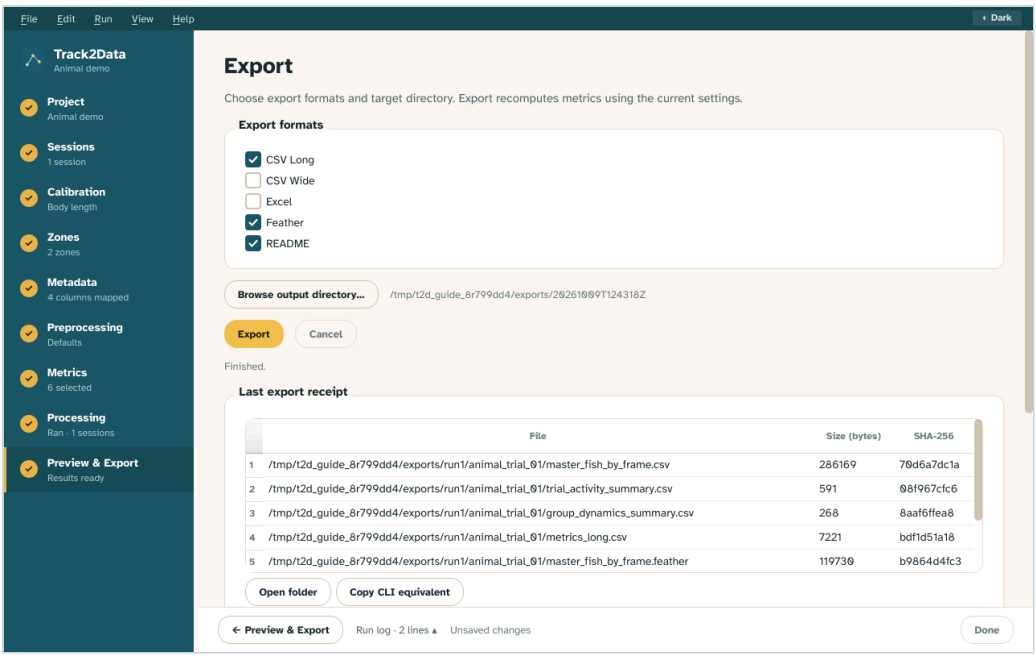


Figure 17. The Export screen, with the file receipt after an export.

1. Tick the formats you want.

Format	Files
CSV Long	`master_fish_by_frame.csv`, `trial_activity_summary.csv`, `group_dynamics_summary.csv`
CSV Wide	`trial_summary_wide.csv` (plus `master_fish_by_frame.csv` when the run has `z_cm` or depth columns)
Excel	`Track2Data_<project>.xlsx` (one sheet per table)
Feather	`master_fish_by_frame.feather`, `trial_activity_summary.feather`
README	provenance record (always written alongside the others)

2. **Browse output directory…** or keep the default.
3. **Export**. Metrics and files are produced again under the current settings, but preprocessing comes from the cache when nothing relevant changed.

The **receipt** lists every file with its size and SHA-256. **Copy CLI equivalent** gives the `track2data run …` command that reproduces this export.

> Very long recordings: Excel limits a sheet to about one million rows, so the per-frame table continues on *Fish by Frame 2*, ... The CSV and Feather files always hold the whole table.

Load the data in R or Python

After an export, the panel at the bottom gives ready-to-paste code for the files just written (R/tidyverse or Python/pandas). **Copy code**, paste, run.

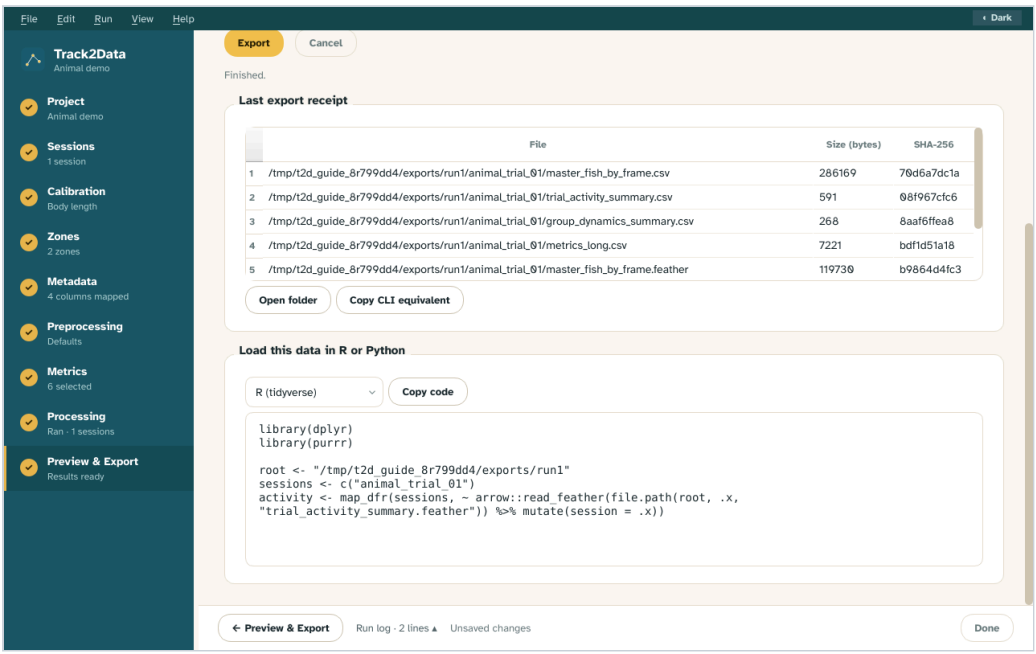


Figure 18. Ready-to-paste R and Python snippets for the files just written.

See Understanding the output files for what the columns mean.

Part II: Reference

Understanding the output files

Each session is written to `<out_dir>/<session_id>/`. Files differ by exporter; the tables are the same.

When the export holds two or more sessions, an extra folder `<out_dir>/all_sessions/` stacks the sessions together: its `master_fish_by_frame.csv`, `trial_activity_summary.csv`, `group_dynamics_summary.csv` and `metrics_long.csv` are the per-session files of the same name joined one after another, with `session_id` saying which session each row came from. Nothing is recomputed. A column that one session lacks (for example because its metric selection differed) is left blank for that session's rows. It also gets a `manifest.json` and `README.md` listing the sessions and their input checksums. Stacking is not the same as pooling: check `PROJECT_SUMMARY.md` and `sessions.csv` for differences in frame rate, group size or calibration first, and treat `session_id` as a grouping factor in your model. A single-session export has no `all_sessions/` folder. Use either the session folders or `all_sessions/`, not both: loading both counts every row twice.

Table	One row per	Contents
`master_fish_by_frame`	session × animal × frame	`time_s`, `x_px`, `y_px`, `was_interpolated` (true where preprocessing filled a gap), `speed_px_s`, `heading_rad`, `main_zone`, `sec_zone`, plus calibrated and metadata columns
`trial_activity_summary`	session × animal	individual-level metrics, plus metadata
`group_dynamics_summary`	session	group-level metrics, plus metadata
`trial_summary_wide`	session × animal	all summary metrics side by side

- **Time bins** (when set on the Metrics screen): summary tables have one row per animal *per bin*, with `bin_index`, `bin_start_s`, `bin_end_s`; `master_fish_by_frame` gets `bin_index`. Metrics that are not meaningful per window keep a single row with empty bin columns.

Column naming

Suffix	Meaning
`_px`	pixels
`_cm`	centimetres (only with a scale; otherwise empty)
`_bl`	body lengths (needs the tracker's body length; independent of the calibration mode and of any physical scale)
`_s`	per second

Things to know

- **Empty `*_cm` columns** mean no pixels-per-unit scale was set, not an error. Use `*_bl`, or set a scale on the Calibration screen.
- **Identity-free sessions** have no per-animal rows for metrics that follow an individual; zone occupancy (Z-1, Z-2, Z-8) is reported pooled over animals, without an `individual_id` column.
- **Missing positions** are empty values, never zeros.
- The README written next to the files records the settings and versions used, and the SHA-256 of each output, so a result can be traced and reproduced.

Definitions, formulas and references for every metric: `docs/METRICS_SPEC.md`.

Troubleshooting

You see	Why	What to do
Next is disabled	A required stage is empty or invalid	Hover over Next: the tooltip names the problem. Check the ✗ / ○ badges in the sidebar
"Session calibration selected but these sessions have no length_unit"	A session has no scale (idtracker.ai never calibrated it, or the other tracker's file records none)	Calibrate it in the idtracker.ai Validator, give **Scale (pixels per cm)** when importing, or switch the calibration mode
"No reader recognised the session folder"	The folder is not an idtracker.ai output	Pick the session folder itself (the one containing `trajectories/`). Supported: idtracker.ai 6.x output (the legacy v5 layout also works); v4 is not supported yet (a v4-looking folder gets a specific message; see sending a v4 sample)
Video column says *Not found*	The video path idtracker.ai recorded does not exist on this computer	Select the session and press **Locate Video…** (Sessions). Metrics do not need the video
READER_NOT_AVAILABLE	A project session was added with software this version of Track2Data does not have	Install a version that has that reader. Track2Data will not read it with a different one, since that could change the numbers
READER_OPTION_MISSING	The software does not record an option (such as frame rate) and none was entered	Add the folder again and fill in the *required* option in the confirm dialog
Session frames / animals show `–`	The folder is still being read	Wait a moment; a failed read is reported in the Run Log

You see	Why	What to do
`*_cm` columns are empty	No pixels-per-unit scale	Use `*_bl` columns or set a scale in Calibration
⚠ on *Sessions*	A session is identity-free	Expected for sessions tracked without identities; see Sessions
Individual metrics missing for a session	That session is identity-free	Untick *Identity-free* only if identities really are stable
"N of M sessions matched" with names listed	Those `session_id` values are not in the metadata CSV	Fix the CSV or the session folder names
A zone warning about resolution	Zones were drawn on a different image size	Redraw them on this session, or confirm the sizes really match
Path length looks far too large	Jumps or identity swaps	Open Preview ▸ Trajectories with *Raw + processed*; keep *Identity switch correction* off
Excel file has *Fish by Frame 2*	The table exceeded Excel's row limit	Use the CSV or Feather file for the complete table
Parallel run slower than sequential	Short sessions; each worker takes time to start	Use 1 worker
macOS says the app is damaged / Windows SmartScreen blocks it	Releases are not signed yet	See `docs/CODE_SIGNING.md`

Still stuck? Open the **Run Log** (View ▸ Toggle Run Log), then an issue on GitHub with its text.

© 2026 Martina Bellio. Released under the MIT licence.